

Anexo 17 — Medicion del alcance del brazo

Archivo: Scripts/explore_arm_reach.py

Para que sirvio:

Herramienta interactiva: abre el visor de MuJoCo con sliders y reporta la posicion de la palma, usada para medir empiricamente el alcance real y definir los limites del curriculum de agarre.

```
1 | """Visor interactivo del DUM4_grab.xml para explorar el alcance de los brazos.
2 |
3 | Abre el MuJoCo passive viewer con el modelo cargado. Mientras esta abierto:
4 | - En la barra LATERAL del viewer (Control panel), puedes arrastrar los sliders de
5 |   cada actuador para mover los joints en tiempo real.
6 | - En la TERMINAL se imprime cada 0.5s la posicion world de las palmas (sites
7 |   grip_R y grip_L) y su offset respecto al baseline (pose en reposo).
8 | - Al cerrar el viewer, se imprime el BOUNDING BOX maximo que las palmas
9 |   alcanzaron durante la sesion (util para definir el cap de la curriculum).
10 |
11 | Tips del viewer (panel izquierdo "Control"):
12 | - Slider 'act_HipBody'      : cadera (rotacion del torso)
13 | - Slider 'act_*ShoulderArm' : hombros
14 | - Slider 'act_*Forearm'     : codos
15 | - Slider 'act_*Wrist'       : muñecas
16 | - Slider 'act_*Lever_Slider' : pinza
17 |
18 | Si no ves el panel Control, presiona la tecla "Tab" en el viewer.
19 |
20 | Uso:
21 |     python Scripts/explore_arm_reach.py
22 | """
23 | from pathlib import Path
24 | import time
25 | import numpy as np
26 | import mujoco
27 | import mujoco.viewer
28 |
29 |
30 | XML_PATH = str(Path(__file__).resolve().parent.parent / "Cuerpo" / "DUM4_grab.xml")
31 | print(f"[setup] Cargando: {XML_PATH}")
32 | model = mujoco.MjModel.from_xml_path(XML_PATH)
33 | data = mujoco.MjData(model)
34 | print(f"[setup] DOFs: {model.nq} qpos, {model.nu} actuadores, dt={model.opt.timestep}s")
35 |
36 | # IDs de los sites de las palmas
37 | grip_R = mujoco.mj_name2id(model, mujoco.mjtObj.mjOBJ_SITE, "grip_R")
38 | grip_L = mujoco.mj_name2id(model, mujoco.mjtObj.mjOBJ_SITE, "grip_L")
39 | if grip_R < 0 or grip_L < 0:
40 |     raise RuntimeError("No encuentre los sites grip_R / grip_L en el XML")
41 |
42 | # Settling - dejar que el robot se asiente en su pose default
43 | mujoco.mj_forward(model, data)
44 | for _ in range(50):
45 |     mujoco.mj_step(model, data)
46 |
47 | baseline_R = data.site_xpos[grip_R].copy()
48 | baseline_L = data.site_xpos[grip_L].copy()
49 | print()
50 | print("[baseline] Posiciones de palma EN REPOSO (world frame):")
51 | print(f"  grip_R: ({baseline_R[0]:+.4f}, {baseline_R[1]:+.4f}, {baseline_R[2]:+.4f})")
52 | print(f"  grip_L: ({baseline_L[0]:+.4f}, {baseline_L[1]:+.4f}, {baseline_L[2]:+.4f})")
53 | print()
54 |
55 | # Bounding box tracker
56 | bbox_R_min = baseline_R.copy()
57 | bbox_R_max = baseline_R.copy()
58 | bbox_L_min = baseline_L.copy()
59 | bbox_L_max = baseline_L.copy()
60 |
61 | print("[viewer] Abriendo viewer. Cerralo cuando termines de explorar.")
62 | print("[viewer] Mové los sliders del panel 'Control' para articular el brazo.")
63 | print()
64 | print(f"{'time':>6s} {'grip_R x':>9s} {'y':>9s} {'z':>9s} | ")
65 | print(f"{'offset_x':>9s} {'offset_y':>9s} {'offset_z':>9s} | {'dist_xyz':>9s}")
66 |
67 | last_print = time.time()
68 | with mujoco.viewer.launch_passive(model, data) as viewer:
69 |     while viewer.is_running():
70 |         mujoco.mj_step(model, data)
71 |         viewer.sync()
72 |         # Update bbox
73 |         pR = data.site_xpos[grip_R]
74 |         pL = data.site_xpos[grip_L]
```

```

75 |         bbox_R_min = np.minimum(bbox_R_min, pR)
76 |         bbox_R_max = np.maximum(bbox_R_max, pR)
77 |         bbox_L_min = np.minimum(bbox_L_min, pL)
78 |         bbox_L_max = np.maximum(bbox_L_max, pL)
79 |         # Print every 0.5s
80 |         now = time.time()
81 |         if now - last_print >= 0.5:
82 |             off = pR - baseline_R
83 |             dist = float(np.linalg.norm(off))
84 |             print(f"{data.time:6.2f} {pR[0]:+9.4f} {pR[1]:+9.4f} {pR[2]:+9.4f} | "
85 |                   f"{off[0]:+9.4f} {off[1]:+9.4f} {off[2]:+9.4f} | {dist:+9.4f}",
86 |                   flush=True)
87 |             last_print = now
88 |             time.sleep(model.opt.timestep)
89 |
90 | # Final summary
91 | print()
92 | print("=" * 70)
93 | print("BOUNDING BOX (durante la exploracion)")
94 | print("=" * 70)
95 | for side, bmin, bmax, base in [("grip_R", bbox_R_min, bbox_R_max, baseline_R),
96 |                               ("grip_L", bbox_L_min, bbox_L_max, baseline_L)]:
97 |     print(f"\n{side} (baseline rest: {base[0]:+.4f}, {base[1]:+.4f}, {base[2]:+.4f})")
98 |     print(f" X world: [{bmin[0]:+.4f}, {bmax[0]:+.4f}] rango {bmax[0]-bmin[0]:.4f}m "
99 |           f"offset: [{bmin[0]-base[0]:+.4f}, {bmax[0]-base[0]:+.4f}]")
100 |     print(f" Y world: [{bmin[1]:+.4f}, {bmax[1]:+.4f}] rango {bmax[1]-bmin[1]:.4f}m "
101 |           f"offset: [{bmin[1]-base[1]:+.4f}, {bmax[1]-base[1]:+.4f}]")
102 |     print(f" Z world: [{bmin[2]:+.4f}, {bmax[2]:+.4f}] rango {bmax[2]-bmin[2]:.4f}m "
103 |           f"offset: [{bmin[2]-base[2]:+.4f}, {bmax[2]-base[2]:+.4f}]")
104 |
105 | print()
106 | print("Para el curriculum (relevante para R, mirrored para L):")
107 | max_out_R = max(0.0, baseline_R[1] - bbox_R_min[1]) # outward = -Y
108 | max_up_R = bbox_R_max[2] - baseline_R[2]
109 | print(f" max OUTWARD (-Y): {max_out_R:.4f}m")
110 | print(f" max UP (+Z): {max_up_R:.4f}m")
111 | print(f" cap simetrico sugerido: {min(max_out_R, max_up_R):.4f}m")
112 |

```